Parallel Contrast Stretch

Anggota:

Ridho Tri Saputra
24060124130122



Deskripsi Singkat Program

Program ini dirancang untuk mendemonstrasikan akselerasi komputasi paralel berbasis Graphics Processing Unit (GPU) menggunakan NVIDIA CUDA dalam melakukan operasi pengolahan citra digital, yaitu Automatic Contrast Stretching. Secara arsitektur, program membagi operasi ke dalam dua tahap kernel utama pada GPU:

1. Tahap pencarian nilai piksel Minimum dan Maksimum menggunakan pola Parallel Tree Reduction.
2. Tahap perhitungan skala kontras secara element-wise di mana setiap thread bertugas memperbarui nilai satu piksel secara independen.

Berdasarkan hasil eksekusi pengujian, pendekatan paralel berbasis GPU ini mencatatkan performa yang sangat tinggi pada citra beresolusi besar:

- Total Piksel Diuji: Hingga resolusi 5184x3456 (jutaan piksel).
- Waktu Eksekusi Kernel: Berjalan sangat cepat di kisaran 0.06 hingga 2.4 milidetik (bergantung pada resolusi).
- Throughput Proses: Mencapai tingkat speedup (percepatan) kernel hingga 13x - 16x lipat lebih cepat dibandingkan pemrosesan sekuensial (CPU).



Persiapan dan Instalasi

Implementasi dan pengujian program dilakukan menggunakan lingkungan komputasi cloud berbasis Google Colaboratory untuk memastikan alokasi GPU yang optimal.

Konfigurasi yang digunakan dalam pengujian adalah:

- GPU: NVIDIA T4 Tensor Core GPU (Google Colab)
- CUDA Toolkit: CUDA 12.x
- CUDA Compiler: nvcc
- Editor/Environment: Google Colab Jupyter Notebook
- Sistem Operasi: Linux (Ubuntu)

Catatan: Jika dijalankan di environment lokal Windows, dibutuhkan instalasi CUDA Toolkit dan MSVC Toolset.



Kompilasi Program

Proses kompilasi dilakukan melalui terminal (atau cell pada Google Colab).
Direktori diarahkan ke folder repositori yang telah diunduh:
cd parallel-contrast-stretch

Program kemudian dikompilasi menggunakan NVIDIA CUDA Compiler (nvcc)
dengan level optimasi -O2:
!nvcc -O2 main.cu -o main

Proses tersebut mengubah source code CUDA berekstensi .cu menjadi executable file main. Berdasarkan pengujian, proses kompilasi berhasil dilakukan tanpa error dan file langsung dapat dijalankan dengan perintah ./main.



Pengujian

```
Cloning into 'parallel-contrast-stretch'...
remote: Enumerating objects: 45, done.
remote: Counting objects: 100% (45/45), done.
remote: Compressing objects: 100% (35/35), done.
remote: Total 45 (delta 18), reused 34 (delta 10), pack-reused 0 (from 0)
Receiving objects: 100% (45/45), 19.66 MiB | 11.41 MiB/s, done.
Resolving deltas: 100% (18/18), done.
/content/parallel-contrast-stretch
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (Use -Wno-deprecated-gpu-targets to suppr

image                  | minmax   |     CPU ms | GPU total ms |  GPU kern ms |  spd total |   spd kern | match?
-----------------------+----------+------------+--------------+--------------+------------+------------+--------
640x426                | atomic   |      0.502 |        0.246 |        0.062 |      2.04x |      8.08x | ok
                       | reduce   |      0.502 |        0.251 |        0.066 |      2.00x |      7.64x | ok
                       | min=28 max=133 (used by all variants)
1280x853               | atomic   |      1.965 |        0.682 |        0.122 |      2.88x |     16.07x | ok
                       | reduce   |      1.965 |        0.705 |        0.152 |      2.79x |     12.91x | ok
                       | min=28 max=133 (used by all variants)
1920x1280              | atomic   |      4.476 |        1.306 |        0.336 |      3.43x |     13.33x | ok
                       | reduce   |      4.476 |        1.268 |        0.281 |      3.53x |     15.91x | ok
                       | min=28 max=134 (used by all variants)
5184x3456              | atomic   |     32.791 |        9.264 |        2.491 |      3.54x |     13.17x | ok
                       | reduce   |     32.791 |        8.992 |        2.115 |      3.65x |     15.50x | ok
                       | min=28 max=134 (used by all variants)

Wrote ./output_cpu/<WxH>.bmp and ./output_gpu/<WxH>.bmp for each sample.
Verify byte-identical with e.g.:  cmp output_cpu/640x426.bmp output_gpu/640x426.bmp
```

Analisis Tahapan Konstruksi Algoritma

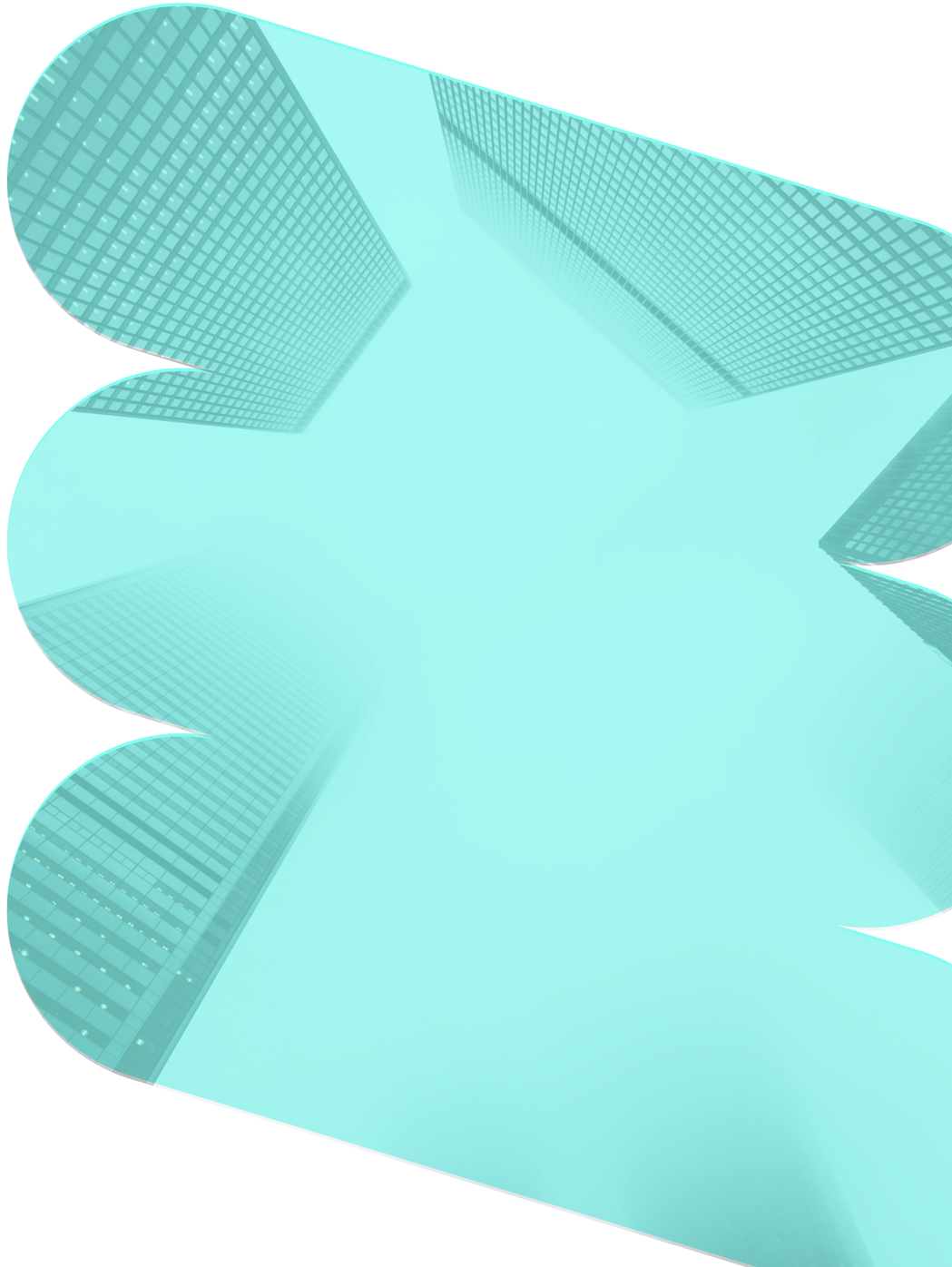
Partioning

1. Identifikasi Pekerjaan Independen: Pada algoritma Contrast Stretching, pemrosesan transformasi setiap piksel bersifat independen.
2. Pemisahan Tugas: Nilai piksel ke-1 (misal di ujung kiri atas) tidak bergantung pada hasil perhitungan piksel ke-1000. Oleh karena itu, operasi subtract (pengurangan nilai minimum) dan scale (perkalian dengan konstanta) dapat dipecah dan dikerjakan secara bersamaan.
3. Ilustrasi Konsep:

Piksel 1 → Hitung (p - min) x scale → Simpan

Piksel 2 → Hitung (p - min) x scale → Simpan (Berjalan Serentak)

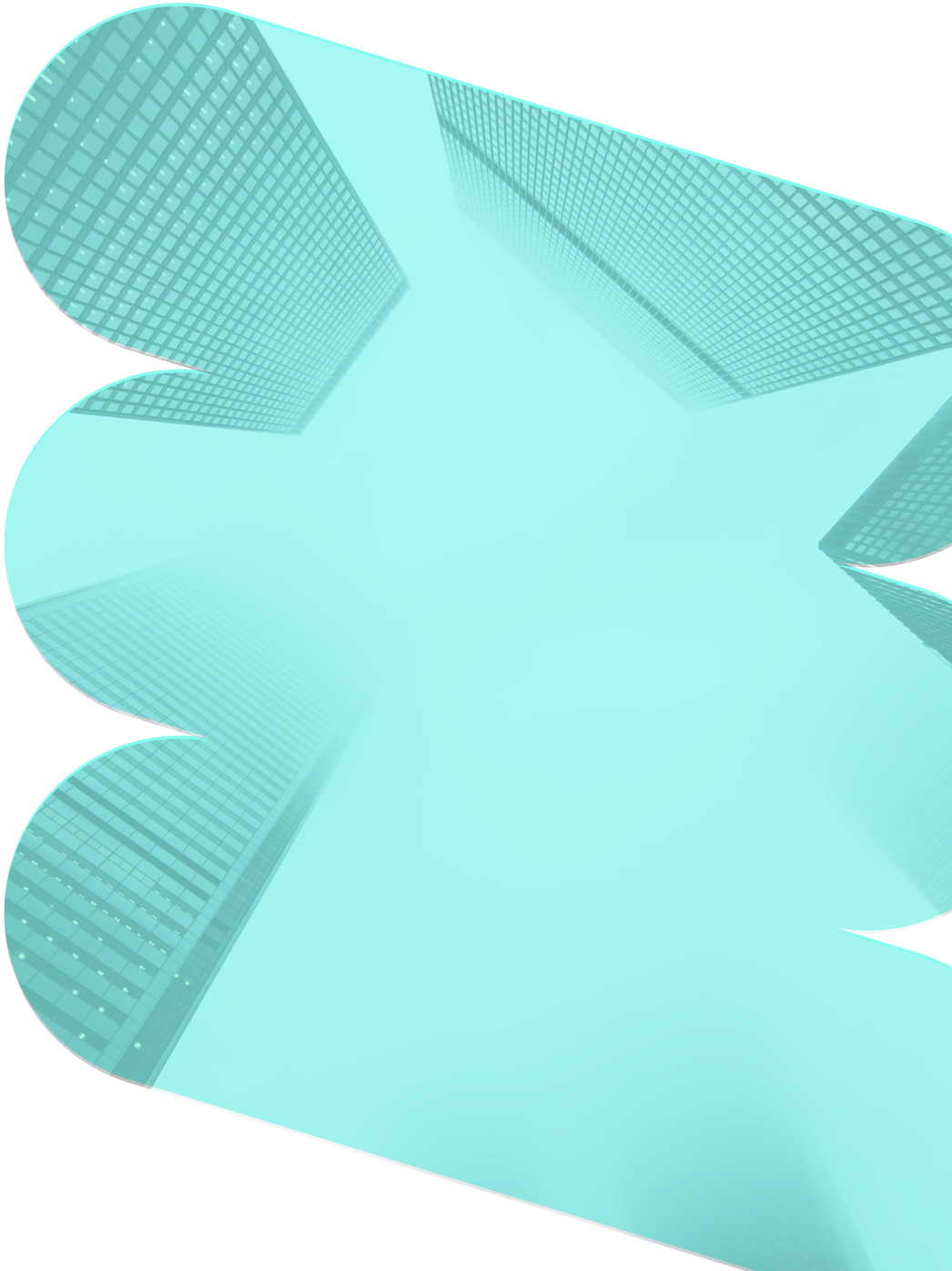
Piksel N → Hitung (p - min) x scale → Simpan



Analisis Tahapan Konstruksi Algoritma

Mapping

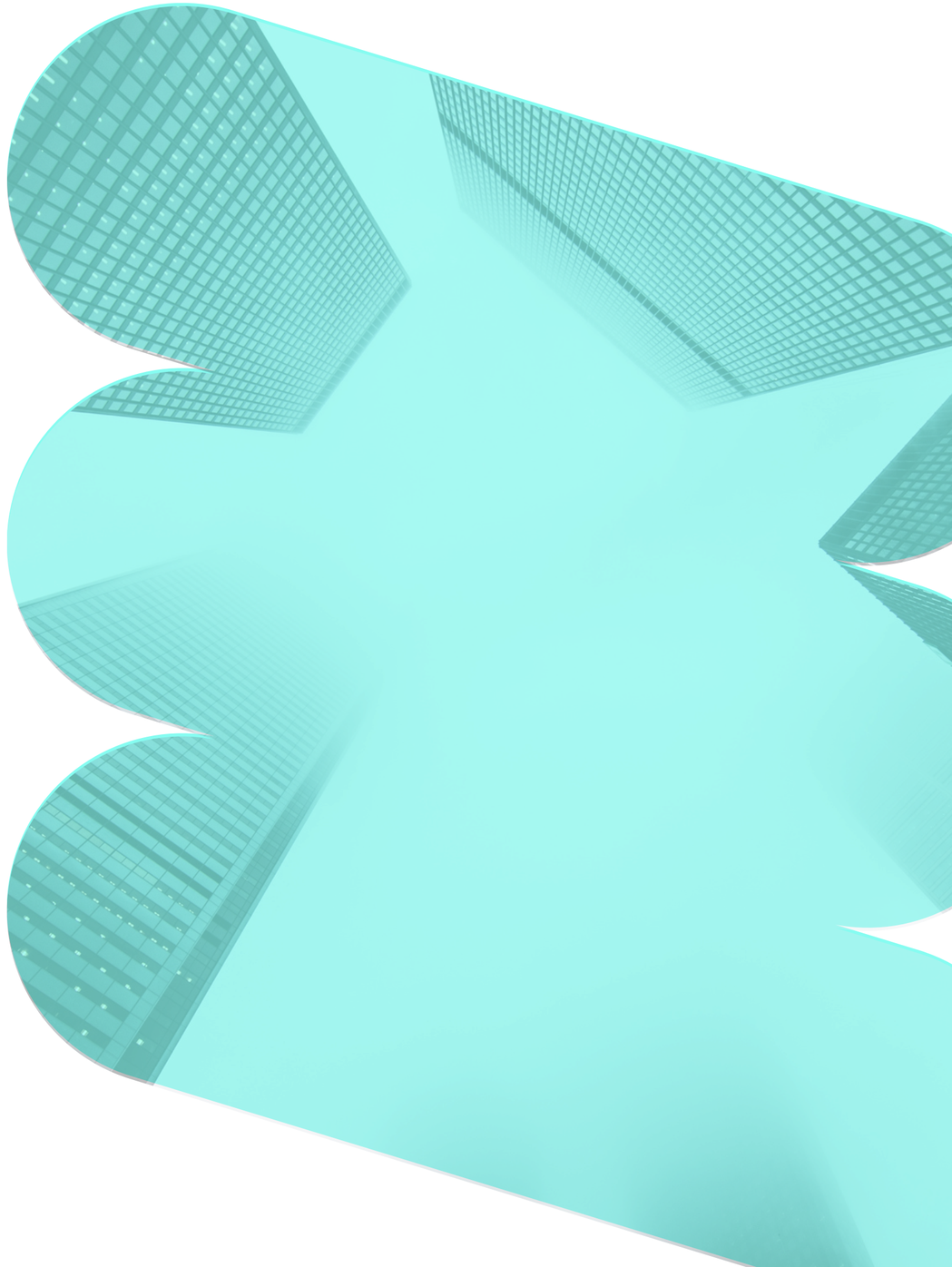
1. Pemetaan Pekerjaan ke Prosesor: Total piksel dari citra (misal resolusi 5184x3456 = 17,915,904 piksel) dipetakan secara langsung ke dalam arsitektur Thread CUDA.
2. Prinsip 1 Thread = 1 Piksel: Setiap Thread di dalam GPU bertanggung jawab mengeksekusi tepat satu piksel.
3. Konfigurasi Blok: Pekerjaan besar ini didistribusikan ke dalam beberapa Block (misal 256 threads per block), dan GPU akan mengeksekusi ribuan block tersebut secara otomatis menyesuaikan ketersediaan Streaming Multiprocessor (SM).



Analisis Tahapan Konstruksi Algoritma

Distribute

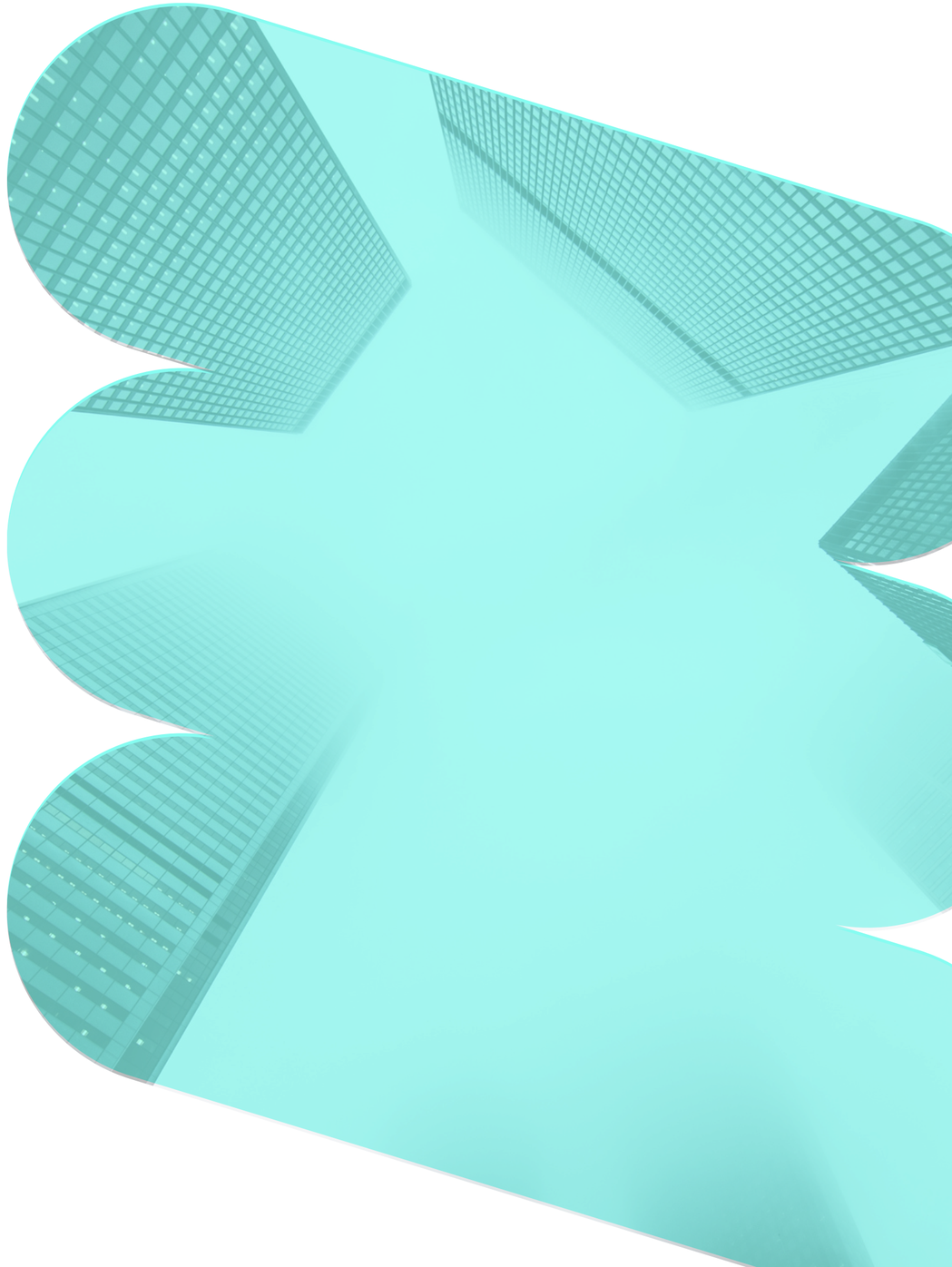
1. Alur Komunikasi CPU (Host) dan GPU (Device): Data mentah tidak bisa langsung diproses oleh GPU, sehingga diperlukan distribusi data melalui bus PCIe.
2. Tahap Distribusi Data:Host ke Device (H2D):
    a. CPU mengirimkan array gambar mentah dan nilai awal inisiasi (min = 255, max = 0) ke memori GPU menggunakan cudaMemcpy().
    b. Proses Device: GPU mengeksekusi kernel reduksi dan kernel element-wise pada memori internalnya yang sangat cepat.
    c. Device ke Host (D2H): GPU mengembalikan array gambar yang kontrasnya sudah dipertajam kembali ke memori CPU menggunakan cudaMemcpy()



Analisis Tahapan Konstruksi Algoritma

Coordinate & Synchronize

1. Koordinasi Shared Data: Pada tahap pencarian nilai min/max piksel, ratusan thread dalam satu block harus bekerja sama. Program menggunakan Shared Memory untuk tree reduction agar proses pencarian cepat tanpa membebani memori global.
2. Pencegahan Race Condition: Untuk memperbarui nilai minimum dan maksimum secara global antar block, program menggunakan operasi atomik (atomicMin dan atomicMax). Ini memastikan tidak ada tabrakan penulisan data saat dua thread mencoba memperbarui nilai di waktu yang sama.
3. Synchronization: Penggunaan __syncthreads() diterapkan dalam Shared Memory reduction untuk memastikan seluruh thread dalam satu blok telah selesai membandingkan piksel sebelum maju ke tahap komputasi berikutnya. CPU juga memanggil cudaEventSynchronize untuk menunggu GPU selesai bekerja 100% sebelum menarik hasilnya.




THANK YOU